

Council Orchestration Console

Prototype Specification Document

Version 0.1 — Prototype Phase

June 2026 | Confidential

Document Revision History

Version	Date	Author	Description
0.1	June 2026	Development Team	Initial prototype specification — derived from SRS v1.1 and tech stack verification

Document Revision History 2

1. Prototype Overview..... 4

 1.1 Purpose 4

 1.2 Scope — What Is In..... 4

 1.3 Scope — What Is Out..... 4

 1.4 Demo Success Criteria 4

2. Tech Stack..... 6

 2.1 Stack Overview..... 6

 2.2 Model Selection for Prototype..... 6

 2.3 Key Architectural Decisions 7

3. How We Are Building It 8

 3.1 Build Sequence 8

 3.2 LangGraph ↔ Node.js Bridge Options..... 8

 Option A — FastAPI sidecar (recommended) 8

 Option B — Child process 8

4. Pre-Development Checklist.....10

 4.1 System Requirements10

 4.2 Accounts and API Keys10

 4.3 Project Setup Steps — In Order10

 4.4 Recommended Folder Structure.....11

5. System Architecture12

 5.1 Architecture Layers.....12

 5.2 Component Interaction — Per Round Lifecycle12

 5.3 LangGraph Graph Structure13

6. Class and Component Design.....14

 6.1 Data Model Classes (Prisma-generated)14

 6.2 Service Layer Classes (Node.js).....14

 6.3 LangGraph State Schema15

7. Complete Context Specification16

 7.1 Council Agent Context Template16

 7.2 Summarizer Context Template16

7.3 Leader Context Template	17
7.4 On-Deck Minimal Briefing Template	17
7.5 Bunkhouse-to-Council Promotion Briefing (FR-055a)	18
7.6 Spotlight Context Template	18
8. Database Architecture.....	19
8.1 Design Decisions.....	19
8.2 Entity Relationships	19
8.3 Prisma Schema Location and Setup.....	19
9. Database Tables.....	20
9.1 agents	20
9.2 projects.....	20
9.3 stages.....	21
9.4 stage_council	21
9.5 rounds	21
9.6 agent_responses.....	22
9.7 running_briefs.....	22
10. API Contract.....	24
10.1 REST Endpoints	24
Agents	24
Projects and Stages.....	24
Rounds and Spotlight.....	24
10.2 Socket.io Events — Server → Client	25
10.3 Socket.io Events — Client → Server	25
11. Environment Configuration	26
12. Error Handling Strategy.....	28
12.1 Error Categories and Recovery	28
12.2 Logging.....	28
13. Demo Seeding Plan	29
13.1 Suggested Demo Scenario.....	29
13.2 Seed Agents (Minimum 5)	29
13.3 Seed Roadmap.....	29
13.4 Demo Walkthrough Script.....	30

1. Prototype Overview

1.1 Purpose

The prototype is a working demo of the Council Orchestration Console that demonstrates the core multi-agent orchestration loop to the client. It is not the full v1 product — it is the fastest path to showing the client that the orchestration concept works end-to-end in a real UI.

1.2 Scope — What Is In

The prototype covers all orchestration logic from the SRS. No RAG, no v2/v3 features.

- Agent library — create, edit, delete agents; assign seat (Council / On-Deck / Bunkhouse / Retired)
- Roadmap builder — define a project, add ordered stages, set round counts, assign Council rosters per stage
- Parallel round execution — all Council agents fire simultaneously via LangGraph parallel nodes
- Bounded context staging — Council agents receive persona + previous round + running brief only
- Summarizer — runs after every round; updates the running brief using Phi-3 Mini via OpenRouter
- Leader synthesis — aggregates all agent responses per round; highlights agreements, disagreements, insights
- Stage auto-transition — system advances to the next stage when round count is reached
- On-Deck minimal briefing — On-Deck agents receive running brief + next stage objective at transition time
- Bunkhouse-to-Council promotion — agents assigned to the new stage roster are promoted and briefed (FR-055a)
- Spotlight — operator can directly address any active Council agent; Leader is bypassed
- Real-time UI — Socket.io pushes live round progress, per-agent status, Leader output, and running brief

1.3 Scope — What Is Out

- RAG document retrieval (FR-047 to FR-053) — excluded from prototype
- v2 Advisor mode (FR-059 to FR-064)
- v3 Picker module (FR-065 to FR-072)
- OpenRouter → Ollama swap (model wrapper is built; the swap itself is deferred)
- Authentication / multi-user (NFR-012 — single operator, no auth required for demo)
- Production packaging / Docker deployment

1.4 Demo Success Criteria

The prototype is considered demo-ready when:

1. An operator can create three or more agents and define a two-stage roadmap with different Council rosters in under 10 minutes.
2. Running a round fires all Council agents in parallel and the UI shows each agent's status in real time.
3. The Summarizer updates the running brief after every round and the brief is visible in the console.
4. The Leader synthesis is displayed after each round with clear structure (agreements, insights, tensions).
5. Stage auto-transition occurs automatically when the round count is reached, with On-Deck agents promoted.
6. Spotlight can be invoked on any active agent and returns a full, unfiltered response without Leader synthesis.

2. Tech Stack

2.1 Stack Overview

Layer	Technology	Version	Role in Prototype
Frontend UI	React + Vite + TailwindCSS	18 / 5 / 3	All UI: agent cards, roadmap editor, round console, Spotlight
Real-time updates	Socket.io client	4.x	Live round progress, per-agent status, Leader streaming
Backend API	Node.js + Express	20 LTS / 4.x	REST endpoints for CRUD, round control, Spotlight trigger
Socket server	Socket.io server	4.x	Push events from orchestration engine to connected clients
Orchestration	LangGraph (Python)	0.2+	Parallel round graph, stage chaining, Summarizer + Leader nodes
Context builder	Custom Python module	N/A	Assembles bounded prompt for each agent type
Database	PostgreSQL	15+	Agents, roadmaps, rounds, responses, running briefs
ORM	Prisma	5.x	Schema migrations, type-safe queries from Node.js
AI runtime	OpenRouter API	Current	All model calls — Council agents, Summarizer, Leader
Model wrapper	Custom callModel() function	N/A	Single interface; swap to Ollama later via AI_MODE env var

2.2 Model Selection for Prototype

All models are called via OpenRouter. Select these for cost-effective demo quality:

Role	Model (OpenRouter ID)	Why
Leader	mistralai/mistral-large	Strong synthesis quality; clearly shows agreements and insights
Council agents	meta-llama/llama-3.1-8b-instruct	Fast, cheap, good persona adherence for parallel calls
Summarizer	microsoft/phi-3-mini-128k-instruct	Matches SRS spec exactly (FR-034); low cost per round
Spotlight queries	mistralai/mistral-large	Same as Leader — full quality response for Spotlight

2.3 Key Architectural Decisions

- LangGraph replaces n8n for the prototype. LangGraph is pure Python — no Docker, no extra services. It gives the same parallel execution and graph-based routing with zero infrastructure overhead.
- Node.js backend bridges to LangGraph via a Python child process or a lightweight FastAPI sidecar. The backend remains JavaScript (developer's proven stack) and LangGraph runs in Python.
- Prisma is added to speed up development. The schema file doubles as living documentation and migrations are one command.
- Socket.io is used from day one, not as an afterthought. The client demo depends on seeing real-time agent execution — this is the most impressive part of the system.
- The model wrapper is built with a mode toggle (AI_MODE=openrouter | ollama) from the start, so the Ollama swap requires no code changes — just a .env change and model name update.

3. How We Are Building It

3.1 Build Sequence

Each step below is a working vertical slice. After each step, the system is runnable and demonstrable.

Step	Deliverable	Estimated Time
1	PostgreSQL schema + Prisma setup + seed script with example agents and roadmap	1 day
2	Node.js / Express API: agent CRUD, project CRUD, stage CRUD, roster assignment endpoints	2 days
3	React UI: Agent Library page with seat management, Roadmap Builder page	2 days
4	LangGraph graph: parallel Council nodes, Summarizer node, Leader node — CLI testable	2 days
5	Node ↔ LangGraph bridge: round execution triggered from API, results written to DB	1 day
6	Socket.io integration: server emits round events, React UI shows live progress	1 day
7	Stage auto-transition: counting logic, On-Deck promotion, Bunkhouse promotion (FR-055a)	1 day
8	Spotlight: endpoint, LangGraph node bypass, direct response to UI	1 day
9	Context builder: bounded prompt assembly for all agent types, tested with real calls	1 day
10	Polish + demo seeding: load a compelling demo scenario, fix rough edges	1 day

3.2 LangGraph ↔ Node.js Bridge Options

Choose one of two integration patterns:

Option A — FastAPI sidecar (recommended)

Run a small Python FastAPI server alongside Node.js. Node.js calls it via HTTP POST /execute-round. LangGraph runs inside FastAPI. Results are written directly to PostgreSQL by Python.

- Pros: clean separation, easy to debug each side independently, LangGraph has full async control
- Cons: one extra process to start (npm run dev runs both via concurrently)

Option B — Child process

Node.js spawns a Python script per round via `child_process.spawn`. The script prints JSON to stdout and Node.js parses it.

- Pros: zero extra server, simple
- Cons: cold start per round, harder to stream events back to Socket.io in real time

Note: Option A is strongly recommended. Real-time streaming of per-agent status requires a persistent connection — a child process makes this significantly harder.

4. Pre-Development Checklist

4.1 System Requirements

Requirement	Minimum Version	Check Command
Node.js	20 LTS	node --version
npm	10+	npm --version
Python	3.11+	python --version
pip	23+	pip --version
PostgreSQL	15+	psql --version
Git	Any	git --version
VS Code	Any	(optional but recommended)

4.2 Accounts and API Keys

7. Create an account at openrouter.ai
8. Generate an API key from the OpenRouter dashboard
9. Add at least \$5 credit (prototype development will use roughly \$1–3)
10. Test the key: run a simple curl against the OpenRouter completions endpoint before writing any code

4.3 Project Setup Steps — In Order

11. Install PostgreSQL locally (postgresql.org/download). Create a database: `createdb council_proto`
12. Create project root folder: `council-proto/`
13. Inside it, create two subfolders: `client/` (React) and `server/` (Node.js + Python)
14. Initialise Node.js in `server/`: `npm init -y`, then install `express`, `socket.io`, `@prisma/client`, `cors`, `dotenv`
15. Initialise Prisma: `npx prisma init` — this creates `prisma/schema.prisma` and a `.env` file
16. Set `DATABASE_URL` in `.env` to your local PostgreSQL connection string
17. Create the Python virtual environment: `python -m venv venv && source venv/bin/activate` (or `venv\Scripts\activate` on Windows)
18. Install Python dependencies: `pip install langgraph langchain langchain-openai fastapi uvicorn httpx psycpg2-binary python-dotenv`
19. Scaffold the React app in `client/`: `npm create vite@latest . -- --template react`, then `npm install socket.io-client tailwindcss`
20. Create a shared `.env.example` file listing all required variables (see Section 11)
21. Commit the empty scaffold to Git before writing any feature code

4.4 Recommended Folder Structure

```
council-proto/
├── client/                                     (React + Vite)
│   ├── src/
│   │   ├── components/
│   │   │   ├── AgentLibrary.jsx
│   │   │   ├── RoadmapBuilder.jsx
│   │   │   ├── RunConsole.jsx
│   │   │   └── SpotlightPanel.jsx
│   │   ├── hooks/
│   │   │   └── useSocket.js
│   │   └── App.jsx
│   └── server/                               (Node.js + Express)
│       ├── prisma/
│       │   └── schema.prisma
│       ├── src/
│       │   ├── routes/
│       │   │   ├── agents.js
│       │   │   ├── projects.js
│       │   │   └── rounds.js
│       │   ├── socket/                       (Socket.io event handlers)
│       │   ├── services/
│       │   │   └── orchestrationBridge.js
│       │   └── index.js
│       └── orchestrator/                     (Python + LangGraph)
│           ├── main.py                       (FastAPI entry)
│           ├── graph.py                     (LangGraph graph definition)
│           ├── nodes/                       (one file per LangGraph node)
│           │   ├── council_node.py
│           │   ├── summarizer_node.py
│           │   ├── leader_node.py
│           │   └── spotlight_node.py
│           ├── context_builder.py
│           └── model_wrapper.py
└── .env
```

5. System Architecture

5.1 Architecture Layers

The prototype has four layers. Each layer communicates only with the layer adjacent to it — the frontend never touches the database directly, and LangGraph never pushes to the UI directly.

Layer	Technology	Responsibilities
Presentation	React + Vite + TailwindCSS + Socket.io client	Agent management, roadmap builder, live round console, Spotlight panel, running brief display
API / Gateway	Node.js + Express + Socket.io server	REST CRUD for agents/projects/stages, round trigger, Spotlight trigger, Socket.io event relay
Orchestration	Python + LangGraph + FastAPI	Parallel agent execution, context assembly, Summarizer, Leader, stage transition logic, state management
Persistence	PostgreSQL + Prisma	Agents, roadmaps, rounds, responses, running briefs — written by both Node.js and Python layers

5.2 Component Interaction — Per Round Lifecycle

The following describes the exact sequence of events from "Operator clicks Start Round" to "Leader synthesis displayed":

22. Operator clicks Start Round in the React UI.
23. React emits a round:start Socket.io event (or calls POST /api/rounds/start).
24. Node.js API creates a Round record in PostgreSQL (status = running), then calls POST /orchestrator/execute-round via HTTP.
25. Node.js emits round:started Socket.io event to all connected clients with the round ID and agent IDs.
26. LangGraph receives the execute-round request. It loads the current project state from PostgreSQL.
27. LangGraph context builder assembles the bounded prompt for each Council agent (persona + previous round + brief).
28. LangGraph fans out to N parallel CouncilAgentNodes — one per active Council agent — all firing simultaneously via asyncio.gather.
29. As each agent call completes, LangGraph writes the response to agent_responses table and emits an agent:complete notification back to Node.js via a callback or webhook.
30. Node.js relays agent:complete events to the frontend via Socket.io in real time.
31. Once all agents complete, LangGraph runs the SummarizerNode: assembles all responses, calls Phi-3 Mini, updates running_briefs table.
32. LangGraph runs the LeaderNode: assembles all responses plus the updated brief, calls Mistral Large, stores synthesis.
33. LangGraph checks TransitionCheckNode: if stage.rounds_done == stage.rounds_total, trigger stage transition.

34. Stage transition runs: promote On-Deck agents, brief Bunkhouse agents assigned to new stage, update seats.
35. LangGraph marks the round complete and returns the synthesis + brief to Node.js.
36. Node.js emits leader:complete and summarizer:complete Socket.io events. UI updates.

5.3 LangGraph Graph Structure

The LangGraph graph is a directed state graph. Each node reads from and writes to the shared OrchestrationState object.

Node	Type	Description
START	Entry	Receives round input — projectId, stageId, roundId, query
load_state	Sequential	Loads Council agents, current brief, previous round from DB
build_contexts	Sequential	Calls ContextBuilder for each Council agent — produces N prompt objects
council_agents (fan-out)	Parallel	N parallel LLM calls — one per Council agent — all fire via asyncio.gather
collect_responses	Sequential	Waits for all N calls, writes responses to DB, emits per-agent events
summarizer	Sequential	Calls Phi-3 Mini with all responses + current brief; updates brief in DB
leader	Sequential	Calls Mistral Large with all responses + updated brief; stores synthesis
transition_check	Conditional	If rounds_done == rounds_total: route to transition_execute; else route to END
transition_execute	Sequential	Promotes On-Deck agents; briefs Bunkhouse agents for new stage; updates seats
spotlight (separate)	Sequential	Separate subgraph: load_state → build_spotlight_context → single_agent_call
END	Terminal	Returns final state: synthesis, updated brief, stage status

6. Class and Component Design

6.1 Data Model Classes (Prisma-generated)

These classes are generated by Prisma from the schema and used throughout the Node.js layer.

Class	Key Properties	Notes
Agent	id, name, personaPrompt, model, seat, createdAt, updatedAt	seat is enum: COUNCIL ON_DECK BUNKHOUSE RETIRED
Project	id, name, query, status, currentStageId, createdAt	status: DRAFT RUNNING PAUSED COMPLETE
Stage	id, projectId, name, orderIndex, roundsTotal, roundsDone, status	status: PENDING ACTIVE COMPLETE
StageCouncil	id, stageId, agentId, seatType, minimalBriefing	Junction: which agents are in Council vs On-Deck per stage
Round	id, stageId, roundNumber, status, query, createdAt, completedAt	status: PENDING RUNNING COMPLETE FAILED
AgentResponse	id, roundId, agentId, response, tokensUsed, latencyMs, createdAt	One record per agent per round
RunningBrief	id, projectId, content, roundCount, updatedAt	One record per project; updated after every round

6.2 Service Layer Classes (Node.js)

Class / Module	Methods	Responsibility
OrchestrationBridge	executeRound(projectId), triggerSpotlight(agentId, query)	HTTP calls to the Python FastAPI orchestrator; relays results to Socket.io
ContextBuilder (Python)	buildCouncil(agent, round, brief) buildOnDeck(agent, stage, brief) buildSpotlight(agent, query, round, brief)	Assembles bounded prompts; enforces the context rules from Section 7
ModelWrapper (Python)	callModel(prompt, model, role) switchMode(mode)	All LLM calls go through here; mode=openrouter or ollama; temperature=0
RoundExecutor (Python)	executeParallel(agents, contexts) collectResponses()	asyncio.gather fan-out; one coroutine per agent; writes responses to DB
SummarizerService	summarize(responses, brief) updateBrief(projectId, content)	Calls Phi-3 Mini; updates running_briefs in DB

Class / Module	Methods	Responsibility
LeaderService	synthesize(responses, brief) formatOutput(synthesis)	Calls Mistral Large; structures output with headings
StageTransitionMgr	check(stage) execute(fromStage, toStage) briefOnDeck() briefBunkhouse()	Counting logic; seat updates; generates minimal briefings for new agents
SpotlightHandler	execute(agentId, query, projectId) bypassLeader()	Runs the spotlight subgraph; explicitly skips Leader node
SocketEmitter (Node.js)	emitAgentStatus() emitLeaderOutput() emitBriefUpdate() emitTransition()	All Socket.io push events; keeps socket logic out of route handlers

6.3 LangGraph State Schema

The shared state object passed between all LangGraph nodes:

```
class OrchestrationState(TypedDict):
    # Identifiers
    project_id: str
    stage_id: str
    round_id: str
    round_number: int
    # Inputs
    query: str
    council_agents: list[dict] # [{id, name, personaPrompt, model}]
    previous_round: list[dict] # [{agentName, response}] or []
    running_brief: str # current brief content
    # Outputs (populated as graph runs)
    agent_contexts: list[dict] # assembled prompts per agent
    agent_responses: list[dict] # [{agentId, agentName, response,
latencyMs}]
    updated_brief: str # brief after Summarizer runs
    leader_synthesis: str # Leader output
    stage_complete: bool
    project_complete: bool
    # Spotlight (only set for spotlight subgraph)
    spotlight_agent_id: Optional[str]
    spotlight_query: Optional[str]
    spotlight_response: Optional[str]
```

7. Complete Context Specification

This section defines exactly what each agent type receives as input. This is the most important implementation constraint from the SRS. Every prompt assembled by ContextBuilder must follow these templates precisely.

Note: The bounded context rule (HC-004, NFR-005): no agent ever receives the full conversation history. Context is always: persona-prompt + previous round verbatim (not all rounds) + running brief. Nothing more.

7.1 Council Agent Context Template

Built for every active Council agent at the start of each round.

```
SYSTEM:
You are {agent.name}.
{agent.personaPrompt}

RUNNING BRIEF (accumulated project knowledge):
{runningBrief.content}
(If empty: "No brief yet. This is the first round.")

PREVIOUS ROUND MESSAGES:
{for each response in previousRound:}
  {response.agentName}: {response.content}
(If round 1: "None. This is the first round of this stage.")

USER:
{project.query}

Respond in your persona. Be specific, critical, and actionable.
Reference prior points where relevant. Do not summarise – analyse.
```

Note: previous round = the immediately preceding round only. Not all rounds. Not all stages. Just the last round's verbatim agent messages.

7.2 Summarizer Context Template

Called automatically after all agent responses are collected. Uses Phi-3 Mini.

```
SYSTEM:
You are a precise technical summarizer. Your only job is to update the running
brief with the key points from the round below.

CURRENT RUNNING BRIEF:
{runningBrief.content}
(If empty: "Start fresh.")

ROUND {roundNumber} – ALL AGENT RESPONSES:
{for each response:}
```



```
=== {response.agentName} ===
{response.content}

USER:
Update the running brief. Extract the 3-5 most important new points from this round.
Append them to the existing brief. Keep the total brief under 800 words.
Return ONLY the updated brief as plain text. No preamble. No commentary.
```

7.3 Leader Context Template

Called after the Summarizer completes. Uses Mistral Large.

```
SYSTEM:
You are the Council Leader. Synthesise the round output below into a structured report.

UPDATED RUNNING BRIEF:
{updatedBrief.content}

ROUND {roundNumber} — ALL AGENT RESPONSES:
{for each response:}
  === {response.agentName} ===
  {response.content}

USER:
Produce a structured synthesis with these sections:

## Key Agreements
Points where agents align. Be specific.

## Notable Tensions
Where agents disagree or pull in different directions.

## Top 3 Actionable Insights
The most immediately useful outputs from this round.

## Blind Spots
Angles that no agent addressed but should be explored.
```

7.4 On-Deck Minimal Briefing Template

Sent to On-Deck agents at stage transition time. This is the ONLY context they receive (FR-025, HC-006). Uses the same model as Council agents.

```
SYSTEM:
You are {agent.name}.
{agent.personaPrompt}

You are currently On-Deck. You will join the active Council in the next stage.

PROJECT BRIEF (what has been established so far):
```

```
{runningBrief.content}

NEXT STAGE OBJECTIVE:
{nextStage.name}

USER:
Acknowledge this briefing with a single paragraph confirming your understanding
and your intended focus for the next stage. You will be called for full responses
soon.
```

Note: The minimal briefing response is stored in `stage_council.minimal_briefing`. It is NOT fed back into subsequent rounds.

7.5 Bunkhouse-to-Council Promotion Briefing (FR-055a)

For agents who are in Bunkhouse but assigned to the new stage's Council roster. Same template as On-Deck (7.4) but the framing changes slightly:

```
You are being activated from the Bunkhouse and joining the Council for the next
stage.
(Replace the On-Deck framing line above with this.)
```

7.6 Spotlight Context Template

Called when the operator directly addresses a specific agent. Leader does NOT process this (HC-007, FR-045).

```
SYSTEM:
You are {agent.name}.
{agent.personaPrompt}

RUNNING BRIEF:
{runningBrief.content}

PREVIOUS ROUND MESSAGES:
{previousRound responses - same as Council template}

USER:
{spotlightQuery}

You are being addressed directly. Give your complete, unfiltered response.
Show your full reasoning, internal debate, and complete position.
The Leader will NOT process this response. This goes directly to the Operator.
```

8. Database Architecture

8.1 Design Decisions

- PostgreSQL is used over SQLite because parallel agent writes (N responses per round) require proper row-level locking. SQLite serialises concurrent writes.
- UUIDs are used as primary keys throughout. This avoids sequential ID enumeration and simplifies future federation.
- All enum types are defined as Prisma enums and enforced at the database level — not just application logic.
- The `running_briefs` table has a `UNIQUE` constraint on `project_id` (one brief per project). Updates use upsert, not insert.
- `stage_council` is a junction table with a `UNIQUE(stage_id, agent_id)` constraint. An agent can be in a stage's roster only once.
- `rounds` has `UNIQUE(stage_id, round_number)` to prevent duplicate round numbers within a stage.
- `agent_responses` has `UNIQUE(round_id, agent_id)` — one response per agent per round.
- `latency_ms` and `tokens_used` in `agent_responses` are nullable. They are populated for observability and future Advisor analysis.

8.2 Entity Relationships

```
Project 1—* Stage
Stage 1—* Round
Stage *—* Agent (via stage_council)
Round 1—* AgentResponse
Agent 1—* AgentResponse
Project 1—1 RunningBrief
```

Cardinality summary:

```
One Project → many Stages (ordered by order_index)
One Stage → many Rounds (counted by round_number)
One Stage → many StageCouncil records (Council + On-Deck agents)
One Round → many AgentResponses (one per Council agent)
One Project → exactly one RunningBrief (upserted after each round)
```

8.3 Prisma Schema Location and Setup

The Prisma schema lives at `server/prisma/schema.prisma`. Run the following commands in order:

37. `npx prisma generate` — generates the Prisma client from the schema
38. `npx prisma migrate dev --name init` — creates the database tables
39. `npx prisma db seed` — runs the seed script (see Section 13)
40. `npx prisma studio` — opens a browser-based DB viewer (useful during development)

9. Database Tables

9.1 agents

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique agent identifier
name	VARCHAR(255)	NOT NULL	Display name (e.g. "Strategic Advisor")
persona_prompt	TEXT	NOT NULL	Full persona instruction for the agent
model	VARCHAR(255)	NOT NULL, DEFAULT llama-3.1-8b-instruct	OpenRouter model ID for this agent
seat	ENUM	NOT NULL, DEFAULT BUNKHOUSE	Current seat: COUNCIL ON_DECK BUNKHOUSE RETIRED
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Creation timestamp
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last update timestamp

9.2 projects

Column	Type	Constraints	Description
id	UUID	PK	Unique project identifier
name	VARCHAR(255)	NOT NULL	Project display name
query	TEXT	NOT NULL	The primary problem question driving all rounds
status	ENUM	NOT NULL, DEFAULT DRAFT	DRAFT RUNNING PAUSED COMPLETE
current_stage_id	UUID	FK → stages.id, NULLABLE	Active stage (null if not yet started)
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Creation timestamp
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last update timestamp

9.3 stages

Column	Type	Constraints	Description
id	UUID	PK	Unique stage identifier
project_id	UUID	FK → projects.id, NOT NULL	Parent project
name	VARCHAR(255)	NOT NULL	Stage display name (e.g. "Problem Framing")
order_index	INTEGER	NOT NULL	Execution order within the project (1-based)
rounds_total	INTEGER	NOT NULL, DEFAULT 3	Number of rounds before auto-transition
rounds_done	INTEGER	NOT NULL, DEFAULT 0	Rounds completed so far
status	ENUM	NOT NULL, DEFAULT PENDING	PENDING ACTIVE COMPLETE
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Creation timestamp

9.4 stage_council

Column	Type	Constraints	Description
id	UUID	PK	Record identifier
stage_id	UUID	FK → stages.id, NOT NULL	Parent stage
agent_id	UUID	FK → agents.id, NOT NULL	The assigned agent
seat_type	ENUM	NOT NULL	COUNCIL ON_DECK — role in this stage
minimal_briefing	TEXT	NULLABLE	Stored briefing response for On-Deck and promoted Bunkhouse agents
		UNIQUE(stage_id, agent_id)	One seat per agent per stage

9.5 rounds

Column	Type	Constraints	Description
id	UUID	PK	Round identifier
stage_id	UUID	FK → stages.id, NOT NULL	Parent stage

Column	Type	Constraints	Description
round_number	INTEGER	NOT NULL	Round number within the stage (1-based)
status	ENUM	NOT NULL, DEFAULT PENDING	PENDING RUNNING COMPLETE FAILED
query	TEXT	NOT NULL	The query sent to agents this round (copied from project.query)
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	When the round started
completed_at	TIMESTAMP	NULLABLE	When the round finished (null while running)
		UNIQUE(stage_id, round_number)	Prevents duplicate round numbers

9.6 agent_responses

Column	Type	Constraints	Description
id	UUID	PK	Response identifier
round_id	UUID	FK → rounds.id, NOT NULL	Parent round
agent_id	UUID	FK → agents.id, NOT NULL	Which agent produced this response
response	TEXT	NOT NULL	Full agent response text
tokens_used	INTEGER	NULLABLE	Total tokens consumed (prompt + completion)
latency_ms	INTEGER	NULLABLE	Wall-clock latency for this model call in milliseconds
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Response timestamp
		UNIQUE(round_id, agent_id)	One response per agent per round

9.7 running_briefs

Column	Type	Constraints	Description
id	UUID	PK	Brief identifier

Column	Type	Constraints	Description
project_id	UUID	FK → projects.id, UNIQUE	One brief per project — UNIQUE enforced
content	TEXT	NOT NULL, DEFAULT empty string	Current accumulated brief text
round_count	INTEGER	NOT NULL, DEFAULT 0	How many rounds the Summarizer has processed
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last Summarizer update timestamp

10. API Contract

All REST endpoints are prefixed `/api`. WebSocket events use Socket.io rooms keyed by `project_id`.

10.1 REST Endpoints

Agents

Method	Endpoint	Body / Params	Response
GET	<code>/api/agents</code>	—	Array of all agents with seat
POST	<code>/api/agents</code>	<code>{ name, personaPrompt, model }</code>	Created agent
PUT	<code>/api/agents/:id</code>	<code>{ name?, personaPrompt?, model? }</code>	Updated agent
DELETE	<code>/api/agents/:id</code>	—	204 No Content
PATCH	<code>/api/agents/:id/seat</code>	<code>{ seat }</code>	Updated agent (seat only)

Projects and Stages

Method	Endpoint	Body / Params	Response
GET	<code>/api/projects</code>	—	All projects
POST	<code>/api/projects</code>	<code>{ name, query }</code>	Created project
GET	<code>/api/projects/:id</code>	—	Project with stages and briefs
PATCH	<code>/api/projects/:id/start</code>	—	Updated project (status: RUNNING)
POST	<code>/api/projects/:id/stages</code>	<code>{ name, orderIndex, roundsTotal }</code>	Created stage
PUT	<code>/api/projects/:id/stages/:sid</code>	<code>{ name?, roundsTotal? }</code>	Updated stage
DELETE	<code>/api/projects/:id/stages/:sid</code>	—	204
POST	<code>/api/projects/:id/stages/:sid/council</code>	<code>{ agentId, seatType }</code>	StageCouncil record

Rounds and Spotlight

Method	Endpoint	Body / Params	Response
POST	<code>/api/projects/:id/rounds/start</code>	—	Round record (status: RUNNING) — rest delivered via Socket.io

Method	Endpoint	Body / Params	Response
GET	/api/projects/:id/rounds/:rid	—	Round with all agent responses + synthesis
POST	/api/projects/:id/spotlight	{ agentId, query }	Spotlight response (synchronous — waits for model call)

10.2 Socket.io Events — Server → Client

Event	Payload	When
round:started	{ roundId, roundNumber, agentIds[] }	Immediately after round is created in DB
agent:thinking	{ agentId, agentName }	When a Council agent call is dispatched
agent:complete	{ agentId, agentName, response, latencyMs }	When an individual agent call returns
summarizer:running	{ }	When Summarizer LLM call is dispatched
summarizer:complete	{ brief }	When brief is updated in DB
leader:running	{ }	When Leader LLM call is dispatched
leader:complete	{ synthesis, roundId }	When Leader synthesis is stored in DB
stage:transition	{ fromStageName, toStageName, promotedAgents[] }	When stage auto-transition executes
project:complete	{ projectId }	When final stage final round completes
round:failed	{ roundId, error }	On any unrecoverable error during round

10.3 Socket.io Events — Client → Server

Event	Payload	When
join:project	{ projectId }	On page load — subscribes client to project room
round:start	{ projectId }	Operator clicks Start Round (alternative to REST POST)
spotlight:invoke	{ projectId, agentId, query }	Operator invokes Spotlight from the console

11. Environment Configuration

All environment variables live in `.env` at the project root. Never commit this file. Commit `.env.example` with placeholder values.

Variable	Example Value	Required	Description
DATABASE_URL	postgresql://user:pass@localhost:5432/council_proto	Yes	Prisma database connection string
OPENROUTER_API_KEY	sk-or-v1-xxxx	Yes	OpenRouter API key for all model calls
AI_MODE	openrouter	Yes	openrouter ollama — controls ModelWrapper routing
OLLAMA_BASE_URL	http://localhost:11434	No	Only needed when AI_MODE=ollama
LEADER_MODEL	mistralai/mistral-large	Yes	Model ID for Leader calls
COUNCIL_DEFAULT_MODEL	meta-llama/llama-3.1-8b-instruct	Yes	Default model for new agents (overridable per agent)
SUMMARIZER_MODEL	microsoft/phi-3-mini-128k-instruct	Yes	Model ID for Summarizer calls
ORCHESTRATOR_URL	http://localhost:8001	Yes	FastAPI orchestrator base URL
ORCHESTRATOR_PORT	8001	Yes	Port for the Python FastAPI sidecar
API_PORT	3001	Yes	Port for the Node.js Express server
CLIENT_ORIGIN	http://localhost:5173	Yes	React dev server origin (for CORS)
NODE_ENV	development	Yes	development production
LOG_LEVEL	debug	No	debug info warn error
MAX_COUNCIL_AGENTS	10	No	Safety cap on parallel agent calls per round

Variable	Example Value	Required	Description
ROUND_TIMEOUT_MS	120000	No	Per-agent call timeout in ms (default 2 minutes)

12. Error Handling Strategy

The prototype must handle errors gracefully during a client demo. A single agent failure should not crash the whole round.

12.1 Error Categories and Recovery

Error Type	Where It Occurs	Recovery Strategy
Single agent timeout	CouncilAgentNode	Mark that agent's response as "[TIMEOUT — no response]". Continue with other agents. Do not fail the round.
Single agent API error	ModelWrapper	Retry once with 2-second delay. If second call fails, record error response. Flag in Socket.io emit. Continue round.
All agents fail	RoundExecutor	Mark round as FAILED. Emit round:failed via Socket.io. Log full error. Do not attempt Summarizer or Leader.
Summarizer fails	SummarizerNode	Keep the previous brief unchanged. Log error. Continue to Leader node (brief will be stale — note this in Leader context).
Leader fails	LeaderNode	Return a "Synthesis unavailable" placeholder. Round is still marked COMPLETE. Agent responses are still stored.
Stage transition fails	StageTransitionMgr	Roll back seat changes (use a DB transaction). Emit stage:transition:failed. Leave current stage active.
Database write fails	Any node writing to DB	Retry once. If second attempt fails, emit round:failed and log. Do not silently swallow DB errors.
OpenRouter rate limit (429)	ModelWrapper	Exponential back-off: wait 5s, retry. If three consecutive 429s, pause round and emit to operator.

12.2 Logging

- All errors are logged with: timestamp, node name, project_id, stage_id, round_id, agent_id (where applicable), and full error message.
- Log levels: DEBUG for context assembly, INFO for round lifecycle events, WARN for retries, ERROR for failures.
- Use the pino library in Node.js and Python's logging module in the orchestrator. Do not use console.log in production code.
- Round timing is logged: round start time, per-agent call start/end, Summarizer start/end, Leader start/end.

13. Demo Seeding Plan

A compelling demo requires pre-loaded data. Run the seed script after migrations to populate the database with a ready-to-run scenario.

13.1 Suggested Demo Scenario

Scenario: "Should our company expand into the Southeast Asian market in the next 18 months?"

This query is specific enough to generate substantive agent disagreement, general enough that any audience can follow it, and produces visibly different viewpoints across agents — which is the core thing to demonstrate.

13.2 Seed Agents (Minimum 5)

Agent Name	Persona Summary	Model	Stage 1 Seat	Stage 2 Seat
Market Strategist	Aggressive growth advocate. Prioritises market share and first-mover advantage over risk. Challenges conservative estimates.	llama-3.1-8b	COUNCIL	COUNCIL
Risk Officer	Systematic risk analyst. Identifies operational, regulatory, and financial risks. Demands quantified downside scenarios.	llama-3.1-8b	COUNCIL	COUNCIL
Operations Director	Execution-focused. Challenges plans that underestimate implementation complexity, hiring timelines, and supply chain requirements.	llama-3.1-8b	COUNCIL	ON_DECK
Financial Analyst	Numbers-driven. Demands ROI models, burn rate projections, and payback period estimates before any strategic claim is accepted.	llama-3.1-8b	ON_DECK	COUNCIL
Customer Advocate	Represents the end customer perspective. Questions whether product-market fit in the home market translates to the target region.	llama-3.1-8b	BUNKHOUSE	COUNCIL

13.3 Seed Roadmap

Stage	Name	Rounds	Council Roster	Purpose in Demo
1	Problem Framing	2	Market Strategist, Risk Officer, Operations Director	Show parallel disagreement, Summarizer building the brief, Leader synthesis
2	Solution Development	2	Market Strategist, Risk Officer, Financial Analyst, Customer Advocate	Show On-Deck promotion (Operations moves out, Financial and Customer join), stage transition in UI

13.4 Demo Walkthrough Script

Use this order when presenting the prototype to the client:

41. Open the Agent Library page — show the five pre-seeded agents with their seats.
42. Open the Roadmap Builder — show the two-stage roadmap with round counts and Council rosters.
43. Start the project — navigate to the Run Console.
44. Start Round 1 — show the three agent tiles light up simultaneously (real-time progress).
45. Point to each agent completing — highlight the latency difference between agents.
46. Show the Summarizer running after all agents complete.
47. Show the Leader synthesis — point to the Agreements / Tensions / Insights structure.
48. Run Round 2 — same stage. Brief now has content; point to how agent responses reference it.
49. Stage auto-transition occurs — show the On-Deck agents joining, the Bunkhouse agent being promoted and briefed.
50. Run one round of Stage 2 — show the new Council roster with four agents.
51. Invoke Spotlight on Risk Officer — ask a sharp follow-up question. Show the direct, unfiltered response.
52. Point to the running brief — show it has accumulated across both stages.